Exercices — Chapitre 4 : Boucles, tableaux et conteneurs

Cours associé : `C04_Boucles_tableaux_conteneurs` · **Fichiers de départ :** `02_Exercices/fichiers/E04/`

> **Méthode :** créez un dossier `E04/` et un fichier par exercice. Avant d'écrire une boucle, notez sur papier sa condition d'arrêt et la valeur de la variable au premier et au dernier tour.

N°	Titre	Niveau	Notions du cours
4.1	Boucles de base	1 — Application	§1
4.2	Prévoir l'affichage	1 — Application	§4.2, §4.3
4.3	Saisie contrôlée	2 — Entraînement	§1, §6
4.4	Premier capteur en défaut	2 — Entraînement	§1.1, §4
4.5	Statistiques de mesures	3 — Problème	§3, §4
4.6	Décoder et valider une matrice	4 — Défi	§5

Niveau 1 — Application directe

Exercice 4.1 — Boucles de base

Objectif : écrire une même répétition avec `for` et avec `while` ; accumuler une somme.

Énoncé :

1. Affichez les nombres de 1 à 20 sur une ligne, séparés par des espaces, avec une boucle `for`.
2. Faites le même affichage avec une boucle `while`.
3. Calculez la somme des entiers de 1 à 100 avec une boucle, puis vérifiez le résultat avec la formule n(n+1)/2.

Résultat attendu :

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Somme : 5050 (formule : 5050)
```

Exercice 4.2 — Prévoir l'affichage

Objectif : suivre l'évolution d'un `std::vector` et des indices d'une boucle.

Énoncé : sans exécuter le programme `ex4_2.cpp`, indiquez ce qu'il affiche, en précisant les indices visités par la boucle. Vérifiez ensuite en compilant.

```cpp
#include <cstddef>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{3, 1, 4};
    v.push_back(1);
    v.push_back(5);

    int total{0};
    for (std::size_t i = 1; i < v.size(); i += 2) {
        total += v[i];
    }

    std::cout << v.size() << ' ' << total << '\n';

    return 0;
}
```

Résultat attendu : le contenu du vecteur après les `push_back`, la liste des indices visités et la ligne affichée.

Niveau 2 — Entraînement

Exercice 4.3 — Saisie contrôlée

Objectif : redemander une saisie tant qu'elle n'est pas valide.

Énoncé : demandez un entier compris entre 1 et 10. Redemandez tant que la valeur est hors de l'intervalle **ou** que la saisie n'est pas numérique. Affichez enfin la valeur acceptée.

Contraintes : messages différents pour une saisie non numérique et pour une valeur hors limites ; si l'entrée est fermée, le programme s'arrête avec le code de retour 1.

Tests : une seule exécution, avec les saisies successives `abc`, `0`, `11`, `7` :

```
Entier entre 1 et 10 : abc
Saisie non numerique.
Entier entre 1 et 10 : 0
Valeur hors limites.
Entier entre 1 et 10 : 11
Valeur hors limites.
Entier entre 1 et 10 : 7
Valeur acceptee : 7
```

Exercice 4.4 — Premier capteur en défaut

Objectif : utiliser `break` pour arrêter une recherche et `continue` pour ignorer des valeurs.

Contexte : une baie contient huit capteurs de température. Un capteur est **en défaut** si sa mesure sort de la plage de -40 à 85 °C inclus. Le relevé est le suivant (indices 0 à 7) :

```cpp
std::array<double, 8> mesures{21.5, 22.0, 23.4, 85.3, 22.8, -41.0, 21.9, 22.4};
```

Énoncé :

1. Parcourez le relevé avec un indice et arrêtez-vous au **premier** capteur en défaut, avec `break`. Affichez son indice et sa mesure, ou `Aucun capteur en defaut` s'il n'y en a pas.
2. Dans une seconde boucle, calculez la moyenne des **seules** mesures valides, en ignorant les autres avec `continue`. Affichez aussi le nombre de mesures valides.

Contraintes : bornes en constantes `constexpr` ; indice de type `std::size_t`.

Résultat attendu :

```
Premier capteur en defaut : indice 3, mesure 85.3
Mesures valides : 6, moyenne : 22.3333
```

Test complémentaire : remplacez `85.3` par `25.0` et `-41.0` par `-40.0`. Le programme doit afficher `Aucun capteur en defaut`, puis 8 mesures valides.

Niveau 3 — Résolution de problèmes

Exercice 4.5 — Statistiques de mesures

Objectif : stocker un nombre inconnu de valeurs dans un `std::vector` et calculer des statistiques.

Énoncé : saisissez des mesures (nombres décimaux) dans un `std::vector` jusqu'à ce que l'utilisateur tape `-1`, qui n'est pas une mesure. Affichez ensuite :

- le nombre de mesures ;
- leur moyenne ;
- le minimum et le maximum ;
- le nombre de mesures **strictement supérieures** à la moyenne.

Si aucune mesure n'a été saisie, affichez `Aucune mesure` et n'effectuez aucun calcul.

Contraintes : on suppose les saisies numériques ; le minimum et le maximum ne sont pas initialisés à 0.

Tests :

Saisies	Affichage attendu
`10` `20` `30` `-1`	3 mesures, moyenne 20, min 10, max 30, 1 au-dessus de la moyenne
`-5.5` `-2` `-8` `-1`	3 mesures, moyenne -5.16667, min -8, max -2, 1 au-dessus de la moyenne
`12` `-1`	1 mesure, moyenne 12, min 12, max 12, 0 au-dessus de la moyenne
`-1`	`Aucune mesure`

Niveau 4 — Défi

Exercice 4.6 — Décoder et valider une matrice

Objectif : exploiter un tableau 2D pour valider le résultat d'une fonction d'affichage.

Contexte : une fonction d'affichage doit produire la lettre **H** sur une matrice de 5 × 5 pixels. Lors d'un test, elle renvoie (1 = pixel allumé) :

```
Ligne 0 : 1 0 0 0 1
Ligne 1 : 1 0 0 0 1
Ligne 2 : 1 1 1 1 1
Ligne 3 : 0 0 0 0 1
Ligne 4 : 1 0 0 0 1
```

Énoncé :

1. Dessinez sur papier la lettre obtenue.
2. La fonction est-elle validée ? Justifiez en citant la ou les cases en défaut (ligne, colonne).
3. Stockez la matrice obtenue dans un tableau 2D et affichez-la avec `#` et `.`.
4. Stockez aussi la matrice **attendue** (un H correct) et écrivez une double boucle qui compare les deux matrices : elle affiche chaque case en défaut, puis le verdict `Validee` ou `Non validee`.

Contraintes : dimensions en constantes `constexpr` ; la comparaison n'utilise aucune case écrite « en dur ».

Résultat attendu (question 4) :

```
Case en defaut : ligne 3, colonne 0
Non validee
```